Introduction

The purpose of this lab was to configure an Apache web server running inside a Docker container to generate log files that could be ingested into Splunk for security monitoring and analysis. Throughout the lab, Apache's logging configuration was examined and modified to write access and error logos to physical logs instead of Docker's standard output. These logs were then verified by generating web traffic and confirming that the requests were successfully recorded. This lab also provided hands-on experience with Docker container management, Apache configuration, Linux command-line, and troubleshooting configuration issues.

Step 1 - Check Where Apache is Writing its Logs

Run: `docker exec -it apache-web cat /usr/local/apache2/conf/httpd.conf | grep -n "CustomLog\|ErrorLog"`

What this command does:

- `docker exec`: Runs a command inside an already running Docker Container
- `-i`: Keeps standard input (STDIN) open. This is commonly used when interacting with containers
- `-t`: Allocates a terminal (TTY), making the output easier to read
- apache-web: The name of the Docker container we're executing the command inside
- `cat`: Displays the contents of a file
- `/usr/local/apache2/conf/httpd.conf`: Apache's primary configuration file
- `|` The pipe operator. It sends the output of the command on the left into the command on the right
- grep: Searches text for matching strings
- `-n`: Shows the line number where each match occurs
- "`CustomLog\|ErrorLog`"
  - CustomLog: defines where access logs are written
  - ErrorLog: defines where Apache error messages are written

```
jake@VivoBook-ASUSLaptop:~$ docker exec -it apache-web cat /usr/local/apache2/conf/httpd.conf | grep -n "CustomLog\|ErrorLog"
311:# ErrorLog: The location of the error log file.
312:# If you do not specify an ErrorLog directive within a <VirtualHost>
317:ErrorLog /proc/self/fd/2
329:    # a CustomLog directive (see below).
346:    CustomLog /proc/self/fd/1 common
352:    #CustomLog "logs/access_log" combined
```

What was found: Apache is not writing to log files. Instead, it is writing to Docker's standard output and standard error streams

The important lines:

- `ErrorLog /proc/self/fd/2`
  - File descriptor 2 is stderr (standard error)
  - Apache sends every error message to Docker's console output
- `CustomLog /proc/self/fd/1 common`
  - File descriptor 1 is stdout (standard output)
  - Apache send every access log entry to Docker's console output

Apache's configuration was inspected to determine where access and error logs were being written. The httpd.conf file showed that CustomLog was configured to write to /proc/self/fd/1

(standard output) and ErrorLog was configured to write to /proc/self/fd/2 (standard error). This configuration is common for Docker containers because Docker automatically captures stdout and stderr, allowing logs to be viewed with the docker logs command.

Step 2 - Backup the Apache Config

Run: `docker exec apache-web cp /use/local/apache2/conf/httpd.conf /usr/local/apache2/conf/httpd.conf.bak`

What this command does;

- `docker exec`: Runs a command inside the apache-web container
- `cp`: Copies a file
- `httpd.conf`: Current Apache config
- `http.conf.bak`: Backup copy in case we need to undo changes

```
jake@VivoBook-ASUSLaptop:~$ docker exec apache-web cp /usr/local/apache2/conf/httpd.conf /usr/local/apache2/conf/httpd.conf.bak
```

Step 3 - Change stdout/stderr Logging to File Logging

Run: `docker exec apache-web sed -i 's|ErrorLog /proc/self/fd/2|ErrorLog "logs/error_log|' /usr/local/apache2/conf/httpd.conf`

What this command does:

- `sed`: edits text
- `-i`: Edit the file in place
- `s|old|new|`: Substitute old text with new text
- Changes Apache error logs from Docker stderr to logs/error_logs

```
jake@VivoBook-ASUSLaptop:~$ docker exec apache-web sed -i 's|ErrorLog /proc/self/fd/2|ErrorLog "logs/error_log"|' /usr/local/apache2/conf/httpd.conf
```

Then run: `docker exec apache-web sed -i 's|CustomLog /proc/self/fd/1 common|CustomLog "logs/access_log" common|' /usr/local/apache2/conf/httpd.conf`

What this command does:

- Changes Apache access logs from Docker stdout to logs/access_logs
- `common` = Apache's basic access log format

```
jake@VivoBook-ASUSLaptop:~$ docker exec apache-web sed -i 's|CustomLog /proc/self/fd/1 common|CustomLog "logs/access_log" common|' /usr/local/apache2/conf/httpd.conf
```

Step 4 - Restart Apache Container

Run: `docker restart apache-web`

What it does:

- Stops and starts the Apache container
- Forces Apache to reload the updated config

```
jake@VivoBook-ASUSLaptop:~$ docker restart apache-web
apache-web
```

Step 5 - Check the Host Log Folder

Run: `ls -l /home/jake/docker/apache/logs`

```
jake@VivoBook-ASUSLaptop:~$ ls -l /home/jake/docker/apache/logs
total 0
```

It can be seen there are no logs in the directory, meaning an error has occurred

Step 6 - Verify the Changes Actually Took Effect

Run: `docker exec apache-web grep -n "CustomLog\|ErrorLog" /usr/local/apache2/conf/httpd.conf`

What this command does:

- `grep`: Searches for text within a file
- `-n`: Prints the line numbers of the matching lines
- This confirms whether our changes to the Apache configuration actually persisted after restarting the container

```
jake@VivoBook-ASUSLaptop:~$ docker exec apache-web grep -n "CustomLog\|ErrorLog" /usr/local/apache2/conf/httpd.conf
Error response from daemon: container dd8b120fabe70aaa7466bea0037ad37ceb1df92acb841e28a32c43605bed6fc6 is not running
```

After attempting to verify, the error message can be seen

Step 7 - Check Error Logs

Run: `docker logs apache-web --tail 50`

What this does:

- `docker logs`: Shows the container's output/error messages
- `apache-web`: Is the container name
- `--tail 50`: Shows only the last 50 lines so it's not overwhelming

```
jake@VivoBook-ASUSLaptop:~$ docker logs apache-web --tail 50
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
[Sat Jun 27 03:26:30.810378 2026] [ssl:warn] [pid 7:tid 7] AH01906: 192.168.1.84:443:0 server certificate is a CA certificate (BasicConstraints: CA == TRUE !?)
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
[Sat Jun 27 03:26:30.813371 2026] [ssl:warn] [pid 7:tid 7] AH01873: Init: Session Cache is not configured [hint: SSLSessionCache]
[Sat Jun 27 03:26:30.813794 2026] [ssl:warn] [pid 7:tid 7] AH01906: 192.168.1.84:443:0 server certificate is a CA certificate (BasicConstraints: CA == TRUE !?)
[Sat Jun 27 03:26:30.815490 2026] [mpm_event:notice] [pid 7:tid 7] AH00489: Apache/2.4.68 (Unix) OpenSSL/3.5.6 configured -- resuming normal operations
[Sat Jun 27 03:26:30.815515 2026] [core:notice] [pid 7:tid 7] AH00094: Command line: 'httpd -D FOREGROUND'
192.168.1.139 - - [27/Jun/2026:03:27:04 +0000] "GET / HTTP/1.1" 200 7121
192.168.1.139 - - [27/Jun/2026:03:27:35 +0000] "-" 408 -
192.168.1.139 - - [27/Jun/2026:03:27:36 +0000] "-" 408 -
192.168.1.139 - - [27/Jun/2026:03:27:36 +0000] "HEAD / HTTP/1.1" 200 -
192.168.1.139 - - [27/Jun/2026:03:29:02 +0000] "HEAD / HTTP/1.1" 200 -
192.168.1.139 - - [27/Jun/2026:03:29:02 +0000] "HEAD / HTTP/1.1" 200 -
192.168.1.139 - - [27/Jun/2026:03:29:03 +0000] "HEAD / HTTP/1.1" 200 -
[Sat Jun 27 17:24:40.436937 2026] [so:warn] [pid 7:tid 7] AH01574: module ssl_module is already loaded, skipping
[Sat Jun 27 17:24:40.437013 2026] [so:warn] [pid 7:tid 7] AH01574: module socache_shmcb_module is already loaded, skipping
AH00526: Syntax error on line 1 of /usr/local/apache2/conf/extra/httpd-ssl.conf:
Cannot define multiple Listeners on the same IP:port
```

The error 'Cannot define multiple Listeners on the same IP:Port' can be seen

Copy the SSL Config Out of the Stopped Container

Run: `docker cp apache-web:/usr/local/apache2/conf/extra/httpd-ssl.conf /tmp/httpd-ssl.conf`

What this does:

- `docker cp`: Copies files between the container and the Ubuntu Laptop
- `apache-web:/path`: Source file inside the container
- `/tmp/httpd-ssl.conf`: Destination on my Ubuntu laptop

Find the Duplicate Listen Lines

Run: `grep -n “Listen” /tmp/httpd-ssl.conf`

What this does:

- `grep`: searches the file
- `-n`: Shows the line number
- This shows where Apache is listening on ports, especially 443

```
jake@VivoBook-ASUSLaptop:~$ grep -n "Listen" /tmp/httpd-ssl.conf
1:Listen 443
```

There is no duplicate Listen line, so we move to the next possible error

Copy the httpd.conf

Run: `docker cp apache-web:/usr/local/apache2/conf/httpd.conf /tmp/httpd.conf`

What this does:

- Copies the main Apache config from the stopped container to my Ubuntu laptop

Search for all Listen Lines

Run: `grep -n “Listen” /tmp/httpd.conf`

```
jake@VivoBook-ASUSLaptop:~$ grep -n "Listen" /tmp/httpd.conf
44:# Listen: Allows you to bind Apache to specific IP addresses and/or
48:# Change this to Listen on specific IP addresses as shown below to
51:#Listen 12.34.56.78:80
52:Listen 80
```

Looking for something that includes SSL config more than once

Run: grep -n "Include" /tmp/httpd.conf

```
jake@VivoBook-ASUSLaptop:~$ grep -n "Include" /tmp/httpd.conf
270:    #   Indexes Includes FollowSymLinks SymLinksifOwnerMatch ExecCGI MultiViews
455:    # (You will also need to add "Includes" to the "Options" directive.)
506:#Include conf/extra/httpd-mpm.conf
509:#Include conf/extra/httpd-multilang-errordoc.conf
512:#Include conf/extra/httpd-autoindex.conf
515:#Include conf/extra/httpd-languages.conf
518:#Include conf/extra/httpd-userdir.conf
521:#Include conf/extra/httpd-info.conf
524:#Include conf/extra/httpd-vhosts.conf
527:#Include conf/extra/httpd-manual.conf
530:#Include conf/extra/httpd-dav.conf
533:#Include conf/extra/httpd-default.conf
537:Include conf/extra/proxy-html.conf
541:#Include conf/extra/httpd-ssl.conf
552:Include conf/extra/httpd-ssl.conf
553:Include conf/extra/httpd-ssl.conf
```

Error: There is a duplicate httpd-ssl.conf

How to fix, run: `sed -i '553s|^Include|#Include|' /tmp/httpd.conf`

What this does

- `sed -i`: Edits the file directly

- `553s|^Include|#Include|` means "*on line 553, replace Include at the start of the line with #Include*"
- This disables the duplicate SSL config include

```
jake@VivoBook-ASUSLaptop:~$ sed -i '553s|^Include|#Include|' /tmp/httpd.conf
```

Then, copy it back by running: `docker cp /tmp/httpd.conf apache-web:/usr/local/apache2/conf/httpd.conf`

```
jake@VivoBook-ASUSLaptop:~$ docker cp /tmp/httpd.conf apache-web:/usr/local/apache2/conf/httpd.conf
Successfully copied 22.5kB to apache-web:/usr/local/apache2/conf/httpd.conf
```

Start Apache Again

```
jake@VivoBook-ASUSLaptop:~$ docker start apache-web
apache-web
```

Apache was started again, and stopped with the same error

Second Error Try

```
jake@VivoBook-ASUSLaptop:~$ docker logs apache-web --tail 30
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
[Sat Jun 27 03:26:30.810378 2026] [ssl:warn] [pid 7:tid 7] AH01906: 192.168.1.84:443:0 server certificate is a CA certificate (BasicConstraints: CA == TRUE !?)
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
[Sat Jun 27 03:26:30.813371 2026] [ssl:warn] [pid 7:tid 7] AH01873: Init: Session Cache is not configured [hint: SSLSessionCache]
[Sat Jun 27 03:26:30.813794 2026] [ssl:warn] [pid 7:tid 7] AH01906: 192.168.1.84:443:0 server certificate is a CA certificate (BasicConstraints: CA == TRUE !?)
[Sat Jun 27 03:26:30.815490 2026] [mpm_event:notice] [pid 7:tid 7] AH00489: Apache/2.4.68 (Unix) OpenSSL/3.5.6 configured -- resuming normal operations
[Sat Jun 27 03:26:30.815515 2026] [core:notice] [pid 7:tid 7] AH00094: Command line: 'httpd -D FOREGROUND'
192.168.1.139 - - [27/Jun/2026:03:27:04 +0000] "GET / HTTP/1.1" 200 7121
192.168.1.139 - - [27/Jun/2026:03:27:35 +0000] "-" 408 -
192.168.1.139 - - [27/Jun/2026:03:27:36 +0000] "-" 408 -
192.168.1.139 - - [27/Jun/2026:03:27:36 +0000] "HEAD / HTTP/1.1" 200 -
192.168.1.139 - - [27/Jun/2026:03:29:02 +0000] "HEAD / HTTP/1.1" 200 -
192.168.1.139 - - [27/Jun/2026:03:29:02 +0000] "HEAD / HTTP/1.1" 200 -
192.168.1.139 - - [27/Jun/2026:03:29:03 +0000] "HEAD / HTTP/1.1" 200 -
[Sat Jun 27 17:24:40.436937 2026] [so:warn] [pid 7:tid 7] AH01574: module ssl_module is already loaded, skipping
[Sat Jun 27 17:24:40.437013 2026] [so:warn] [pid 7:tid 7] AH01574: module socache_shmcb_module is already loaded, skipping
AH00526: Syntax error on line 1 of /usr/local/apache2/conf/extra/httpd-ssl.conf:
Cannot define multiple Listeners on the same IP:port
[Sat Jun 27 17:43:10.219012 2026] [so:warn] [pid 7:tid 7] AH01574: module ssl_module is already loaded, skipping
[Sat Jun 27 17:43:10.219100 2026] [so:warn] [pid 7:tid 7] AH01574: module socache_shmcb_module is already loaded, skipping
AH00526: Syntax error on line 1 of /usr/local/apache2/conf/extra/httpd-ssl.conf:
Cannot define multiple Listeners on the same IP:port
jake@VivoBook-ASUSLaptop:~$ docker ps -a --filter name=apache-web
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS                          PORTS     NAMES
dd8b120fabe7   httpd:latest   "sh -c 'echo \"Includ…"   14 hours ago   Exited (1) About a minute ago             apache-web
```

What is likely happening: The container startup command is adding this line every time it starts: Include conf/extra/httpd-ssl.conf

Attempt at Fixing

Run: `sed -i '/^Include conf\/extra\/httpd-ssl.conf$/d' /tmp/httpd.conf`

What it does:

- `sed -i`: Edits the file directly
- `'/^Include conf\/extra\/httpd-ssl.conf$/d'` : Deletes every active line that exactly says 'Include conf/extra/httpd-ssl.conf '

```
jake@VivoBook-ASUSLaptop:~$ sed -i '/^Include conf\/extra\/httpd-ssl.conf$/d' /tmp/httpd.conf
```

Copy the File Back

Run: `docker cp /tmp/httpd.conf apache-web:/usr/local/apache2/conf/httpd.conf`

```
jake@VivoBook-ASUSLaptop:~$ docker cp /tmp/httpd.conf apache-web:/usr/local/apache2/conf/httpd.conf
Successfully copied 22.5kB to apache-web:/usr/local/apache2/conf/httpd.conf
```

Start Apache again

```
jake@VivoBook-ASUSLaptop:~$ docker start apache-web
apache-web
jake@VivoBook-ASUSLaptop:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS         PORTS                                                                                  NAMES
dd8b120fabe7   httpd:latest   "sh -c 'echo \"Includ…"   14 hours ago   Up 4 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp   apache-web
```

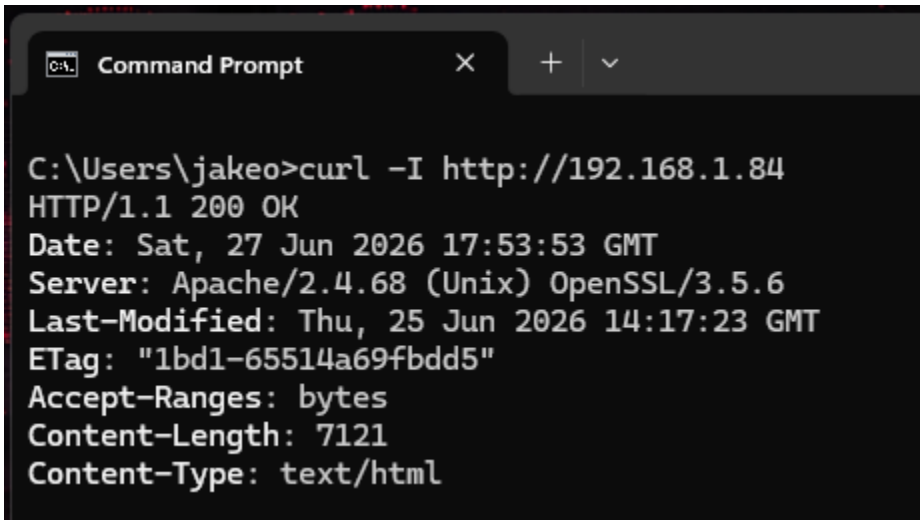
Success!

Check Log Files are Present

Run: `ls -l /home/jake/docker/apache/logs`

```
jake@VivoBook-ASUSLaptop:~$ ls -l /home/jake/docker/apache/logs
total 8
-rw-r--r-- 1 root root   0 Jun 27 13:50 access_log
-rw-r--r-- 1 root root 879 Jun 27 13:50 error_log
-rw-r--r-- 1 root root   2 Jun 27 13:50 httpd.pid
```

Verify Everything is Working



A curl request is sent from a Windows desktop to the Apache Docker

```
jake@VivoBook-ASUSLaptop:~$ tail -f /home/jake/docker/apache/logs/access_log
192.168.1.139 - - [27/Jun/2026:17:52:34 +0000] "GET / HTTP/1.1" 200 7121
192.168.1.139 - - [27/Jun/2026:17:53:26 +0000] "-" 408 -
192.168.1.139 - - [27/Jun/2026:17:53:27 +0000] "GET / HTTP/1.1" 200 7121
192.168.1.139 - - [27/Jun/2026:17:53:53 +0000] "HEAD / HTTP/1.1" 200 -
```

The Apache Docker receives the request and logs it

Conclusion

Troubleshooting and Challenges

1. Apache Logs Weren't Being Written to Files

   Initially, the mounted log directory only contained the httpd.pid file, while the expected access_log and error_log files were missing. However, the docker logos apache-web command displayed Apache access logs, indicating that Apache was sending its logs to Docker's standard output rather than writing logs files

   To verify this, the Apache configuration file (`httpd.conf`) was examined. The configuration showed:

   ➔ `CustomLog /proc/self/fd/1 common`

   ➔ `ErrorLog /proc/self/fd/2`

   These directives instructed Apache to write logs to Docker's standard output and standard error streams instead of physical log files

   The configuration was modified so Apache would write to:

   ➔ `logs/access_logs`

   ➔ `logs/error_logs`

   This allowed the host system to access the log files through the Docker volume mount, making them suitable for ingestion into Splunk

2. Apache Failed to Start After Configuration Changes

   After modifying the logging configuration, the Apache container exited immediately after startup

   The container logs were examined using `docker logs apache-web`

   The logs reported the following error:

   `Cannot define multiple Listeners on the same IP:port`

   Further inspection of the Apache configuration files revealed that the SSL configuration file (httpd-ssl.conf) had been included multiple times within httpd.conf. This caused Apache to attempt to create multiple listeners on TCP port 443

   The duplicate `Include conf/extra/httpd-ssl.conf` directives were removed, eliminating the conflicting listener definitions

3. Verification

   After correcting the Apache configuration, the container started successfully. The mounted log directory contained:

   ➔ `access_log`

   ➔ `error_log`

```
➔ httpd.pid
```

A test request was generated using curl, and the request immediately appeared in access_log, confirming that Apache was successfully writing logs to files that can now be monitored in Splunk (Will be set up in a future lab).

Lessons Learned

Throughout the lab, a deeper understanding was developed of how Apache logging functions within a Docker environment and how those logs can be prepared for ingestion into a SIEM platform. One of the most important lessons learned was that containerized applications often write logs to standard output (`stdout`) and standard error (`stderr`) rather than to traditional log files. Because of this, simply mounting the Apache log directory wasn't enough to make log files available to Splunk.

The importance of validating application configurations before making assumptions was also reinforced. Inspection of the Apache configuration file made it possible to identify where logos were being written and modify the configuration to output access and error logs to physical files. During troubleshooting, duplicate SSL configuration entries were found to prevent Apache from starting successfully. Reviewing the container logs made it possible to identify the error, isolate the cause, and correct the configuration.

The value of verifying each configuration change individually was also emphasized. Testing the application after each modification made it easier to identify problems quickly and ensured that the final logging configuration functioned as expected before integrating the logs into Splunk.

Skills Gained

- Configured and managed an Apache web server running inside a Docker container
- Modified Apache configuration files to redirect logging from Docker standard output to traditional access and error log files
- Used Docker commands (`docker exec, docker logs, docker cp, docker restart,` and `docker inspect`) to administer and troubleshoot containerized applications
- Diagnosed and resolved Apache startup failures by analyzing container log output and configuration files
- Verified Docker volume mounts and confirmed successful log generation on the host operating system
- Generated web traffic using HTTP requests to validate Apache logging functionality

- Prepared Apache log files for ingestion into a Security Information and Event Management (SIEM) platform
- Applied a structured troubleshooting methodology by identifying issues, validating assumptions, implementing configuration changes, and verifying successful operation
- Strengthened Linux command-line proficiency through file management, text searching, log monitoring, and configuration editing
- Improved understanding of how web server logs are generated, stored, and collected within a containerized environment